

CHART

The testbed lexicon: two layers, five edges, four modalities, one linked record

CHART is the Configurable Human-AI Research Testbed. Most teams describe how they work together and then hope. CHART writes it down as a graph and makes the software enforce it.

The page states what the testbed is. This file states what each of its words means, what each is not, and how to decide whether it applies to a case in front of me. Where the page and the book chapter and the code say the same thing, the entry says it once. Where they do not, the disagreement sits in the entry, in a marked box, because those are the entries worth having. Three of them turn out to matter: the graph is called acyclic and the synchrony edge is not, the page names five edge types and the implementation carries seven, and the two page sets split the testbed into a research object and a piece of software without saying which one the reader is holding.

The cluster's shared vocabulary is in `hac_shared`. Complementarity, interdependence, authority boundary, trace, over-reliance and trust calibration live there and are used here without restating.

Contents

1	Active	2
1.1	The architecture	2
1.2	The five dependency types	5
1.3	The four modalities	7
1.4	The record	9
2	Working	11
3	Peripheral	12
4	Four sources, four emphases	13
5	Next	13

1 Active

1.1 The architecture

Configurable Human-AI Research Testbed

“Existing platforms fix the team: CybORG and the CAGE challenges put the human outside the loop as an evaluator”

website, CHART

Definition: An experimental platform in which the organization of a human-agent team is specified as data, enforced at execution time, and varied between conditions.

Distinguishes it from: A cyber-defense environment such as CybORG, which fixes the team and varies the scenario. CHART holds the scenario and varies the team. The Team Defense Game sits between them: it brought the human inside the loop, with one fixed coordination pattern.

Operational test: Between two conditions, is the difference expressible as a change to the team specification, with the scenario file untouched? If it needs a new scenario, the platform is not configurable in this sense.

In my project: The chapter states the claim as three advances over prior testbeds: a graph-based framework for interdependence, the ability to vary role assignment, authority boundaries and communication channels inside one architecture, and multimodal interaction from natural-language explanation to approval workflow.

taskwork interdependence

“The inner layer is taskwork interdependence: a directed acyclic graph over teammates, human and AI, whose edges say who must coordinate with whom, under what condition, in what order.”

website, CHART, two layers and a recorder

Definition: The structural layer. What the team is required to do together, written as typed edges over the teammates, and enforced before an action takes effect.

Distinguishes it from: Teamwork modalities, the outer layer. Taskwork interdependence decides what is permitted; modalities decide how the permission is negotiated. The clean test is that removing a modality changes how the team communicates and removing an edge changes what the team can do.

Operational test: Turn off every interface and leave the graph. Is anything still constrained? Whatever survives is taskwork interdependence.

In my project: Section 2.1 of the chapter, and the object it names is a `networkx.MultiDiGraph` built by the `DAG` class in `blueAgent/00api.py`. Every constraint is edge data, so the layer is literally a serializable object rather than a description.

teamwork modalities

“The outer layer is teamwork modalities: the interaction mechanisms through which that coordination actually happens”

website, CHART, two layers and a recorder

Definition: The four interfaces through which coordination is carried out: pre-task configuration, approval, explanation, communication.

Distinguishes it from: Features of the AI teammate. A modality belongs to the testbed and is present whatever agent is plugged in. Whether the agent can produce a faithful rationale is a property of the agent; whether the rationale reaches a panel that logs consultation is a property of CHART.

Operational test: Swap the agent. Does the mechanism survive? Then it is a modality.

In my project: Section 2.2. The chapter’s argument for putting explanation here is that agent paradigms each cover part of the requirement and none covers all of it, so the channel has to be owned by the platform rather than left to the agent.

behavioral and preference data collection

“Underneath both sits data collection, instrumenting every step with time-stamped traces.”

website, CHART, two layers and a recorder

Definition: The third architectural layer, sitting under the other two, which time-stamps every event in both and links events across streams.

Distinguishes it from: Logging. The distinguishing property is that an entry in one stream can be resolved against an entry in another, which is what makes a sequence recoverable.

Operational test: Pick an approval. Can you retrieve the explanation consultation that preceded it and the chat message that preceded that, without matching on wall-clock time by hand?

In my project: Section 2.3 and Table 2 of the chapter, which sets out the streams, their granularity, and their two uses. In the running system each stream is a database model of its own, one per round of commands, one per submitted round action, one per chat message, one per agent selection, with the graph configuration, the per-episode dependency map and the approval history carried as structured fields on the room.

the dependency graph

“Nodes are agents and pools; edges are typed.”

website, CHART, five dependency types

Definition: A directed multigraph whose nodes are teammates and pool nodes and whose edges each carry a `dependency_type` and that type’s parameters. The direction of an edge says whose action constrains or informs whom.

Distinguishes it from: An organization chart. A chart names reporting lines and is read by people. This graph is read by the environment on every step and decides whether a submitted action executes.

Operational test: Submit a forbidden action. If it executes anyway and the violation only appears in a report, the graph is documentation.

In my project: Built through `add_agent`, `add_pooling_node` and five `add_*_edge` calls, and shown to participants after configuration as a work plan: which tasks are sequential, which run

in parallel, which need approval or synchronization.

The graph is called acyclic and one edge type makes a cycle

The page, the artifact design page and the chapter all say directed acyclic graph, and the design page rests an argument on it: because the structure is a DAG it can be walked and queried rather than only read.

`DAG.add_synchrony_edge` in `blueAgent/00api.py` adds two edges, one in each direction, with the same `sync_group`. Its own docstring says so: adds bidirectional synchrony edges. Two agents in a synchrony group are a two-cycle, and the container is a `MultiDiGraph`, which does not enforce acyclicity in the first place.

This is not a bug in the running system, because synchrony is checked symmetrically by design. It is a naming problem with a consequence: any analysis that assumes a topological order over the whole graph is unsound on a session that used synchrony. Either the acyclicity claim is scoped to the constraining edge types, control, temporal and pool, or the synchrony pair is represented as one undirected edge and the walk skips it.

pool node

Definition: A non-agent node that accumulates contributed actions and issues one composite action when the threshold is met, then clears. The API documentation calls these pseudo-agents and names two, `action_pool` and `obs_pool`.

Distinguishes it from: A teammate. A pool node makes no decisions and holds no observation of its own. It exists so that pooling logic lives in one place instead of as conditionals scattered through the environment.

Operational test: Can it be an approver, or hold a policy? No to both means it is a pool node.

In my project: `add_pooling_node(node_name, pool_threshold, composite_action)`. The threshold is stored as a set of `(agent, action, target)` tuples rather than as a count, so the default form is a named set of required contributions and the k-of-n quorum is the variant.

the configuration is a contract

Definition: The chapter's word for what the pre-task configuration produces. The agent is set to operate within stated boundaries and the humans calibrate their expectations against the same statement.

Distinguishes it from: A briefing. A briefing tells people what to expect. A contract is the object the environment consults on every step, so a violated expectation is a blocked action and not a misunderstanding.

Operational test: Is the artifact the participants read the same artifact the environment executes? If two documents exist, one of them is a briefing.

In my project: The graph preview shown after configuration is generated from the same edge data the environment enforces. That identity is the reason a participant's expectation can be treated as a manipulation rather than as noise.

1.2 The five dependency types

Each type is a coordination pattern that shows up in real security operations, with required inputs and optional inputs. The chapter's Table 1 is the authority on the inputs; the API is the authority on which are implemented.

control dependency

Definition: See `hac_shared`. Required inputs: an approver, a requester, and the set of actions requiring authorization, optionally scoped to targets. Optional inputs: escalation chain, risk-tier gating, timeout policy, batch approvals, per-target scoping, audit level.

Distinguishes it from: Temporal, which also blocks. Control is cleared by another agent's decision; temporal is cleared by a prior action.

Operational test: Who can clear the block?

In my project: `add_control_edge(startingAgent, endingAgent, approval_required)`, where `approval_required` is a list of action names. The set of actions requiring approval is therefore edge data, which is what makes it a one-line manipulation.

timeout policy

"a timeout policy that either auto-approves or auto-denies"

website, CHART, control

Definition: What happens to a pending action when the approver does not act within the escalation delay. Auto-approve emphasizes speed; abort emphasizes safety.

Distinguishes it from: The escalation chain, which decides who is asked next. The timeout decides what happens when nobody answers.

Operational test: Let the timer run out and do nothing. Whichever way the action resolves is the policy, and it is the single field that decides whether the human is in the loop or on it.

In my project: Named on the artifact use page as the fifth of the five single named changes an experimenter can make, and the chapter states the reason it is worth varying: it converts the throughput-versus-safety trade-off into an empirical variable.

pool dependency

Definition: See `hac_shared`. Required inputs: the contributors, the pool node, the contributing actions, the threshold, and the composite action. Optional inputs: contribution weights, a sliding time window, k-of-n quorum variants, reset policy on fire or on timeout, per-host aggregation, inhibitor conditions.

Distinguishes it from: Synchrony. Both require several agents to act. Pool accumulates across turns until the threshold is met; synchrony requires the actions inside one window and drops them otherwise.

Operational test: If the second contribution arrives three turns later, does the composite action still fire? Yes is pool. No is synchrony.

In my project: `add_pool_edge(startingAgent, endingAgent, contribute_actions)` into a pooling node. The environment checks each pool buffer after every step and fires the composite

action once when the buffer meets the threshold, then empties it.

synchrony dependency

“designated actions must occur in the same turn or window to take effect”

website, CHART, five dependency types

Definition: A requirement that the members of a sync group act inside one window for any of their actions to take effect. Coordinated shutdowns to cut off lateral movement. Optional inputs: a tolerance window, k-of-n synchrony, ordered subgroups, retry and backoff, and precedence against other dependency types.

Distinguishes it from: Temporal, which constrains order and not simultaneity. Synchrony has no order inside the window.

Operational test: Does acting first and alone accomplish nothing? Then the constraint is synchrony rather than sequence.

In my project: `add_synchrony_edge(startingAgent, endingAgent, sync_group)`, which writes the edge in both directions. See the acyclicity box above.

temporal dependency

“strict ordering. Forensic analysis before containment, so evidence survives; scanning before remediation.”

website, CHART, five dependency types

Definition: A requirement that named prerequisite actions complete before named dependent actions may execute. Optional inputs: validity or expiry windows on the prerequisite, evidence requirements such as a hash or a log identifier, per-target linkage, cooldowns, and exception roles allowed to bypass.

Distinguishes it from: A scenario script, which decides what happens when. A temporal edge decides what is permitted when, and the agent is free to do nothing at all.

Operational test: Submit the dependent action first. If it is refused rather than reordered, the edge is temporal.

In my project: `add_temporal_edge`, taking the two agents plus `prerequisite_actions` and `dependent_actions`, both as lists of tuples. The chapter’s account of what this makes studyable is worth keeping: agents accelerate a workflow and cannot circumvent its required sequence.

informational dependency

Definition: See `hac_shared`. Required inputs: a source, a sink, and an observation filter, a predicate or mapping from an observation to allowed or redacted. Optional inputs: redaction level, meaning masking against aggregation, time-gated access, role or clearance conditions, event-triggered reveals, sampling rate caps, audit granularity.

Distinguishes it from: Communication, which is what a teammate sends.

Operational test: With everyone silent, does one agent see less?

In my project: `add_informational_edge`, taking the two agents plus an `obs_filter` that may be passed as a callable or as a string expression. The team configurator composes the filters

into each agent's observation function before the mission starts, so an agent's observation space is a product of the graph and not of its own code.

Five on the page, seven in the implementation

The research page, the artifact design page and the chapter all name five types.

`api.py` defines a sixth, `add_temporal_informational_edge`, writing `dependency_type="temporal_informational"` with a `turn_count` and a list of dependent observations. It models delayed information sharing: what a teammate may see, but not yet.

`apiSpec.md` lists a seventh, `trigger`, on the edge from a pool node out to the environment, carrying `approval_required` for the composite action. That is what allows a control constraint to sit on a collective action rather than on an individual one.

Both are compositions rather than new primitives, which is a defensible reason to keep the taxonomy at five. It is not a reason to leave them undocumented. A delayed reveal and an approval on a composite action are the two configurations a security operations centre would ask for first, and neither is reachable from the page.

1.3 The four modalities

pre-task configuration

"A lobby where teams assemble, roles are assigned, the AI teammate's capability profile is chosen, and the dependency graph is previewed before anyone starts."

website, CHART, the modalities

Definition: Everything that happens before the mission: team assembly, role assignment, agent selection, graph construction, and the work-plan preview.

Distinguishes it from: Instructions. Configuration writes the object the session runs under; instructions describe it.

Operational test: Would changing this screen change what the environment permits? Yes puts it in configuration.

In my project: Who does the configuring is itself the manipulation. An experimenter-set structure studies how people react to an imposed one; a participant-built structure studies how they choose to organize themselves. Both are supported and the difference is a condition.

capability profile

"how aggressive or cautious it is in raising alerts and taking action"

artifacts, CHART, Use, configure the team

Definition: The behavioural setting chosen for an AI teammate before the mission, which sets the human's expectation about how the agent will act.

Distinguishes it from: The authority boundary. The profile is what the agent will attempt. The boundary is what it is permitted to complete. A cautious profile inside a loose boundary and an aggressive profile inside a tight one produce different traces and can produce the same outcome.

Operational test: Does changing it alter the agent’s proposals, or the fate of its proposals? Proposals means profile. Fate means structure.

In my project: Chosen in the agent-selection screen and recorded as an input in the Input-Process-Outcome record, which is what makes perceived reliability a between-condition variable rather than an uncontrolled one.

approval queue

“Actions on #control edges enter a queue visible to their supervisor, who can grant, deny, or ask for an explanation.”

website, CHART, the modalities

Definition: The interface that realizes a control edge. Pending actions wait in it, and the supervisor grants, denies, or requests an explanation.

Distinguishes it from: A notification feed. A queue holds an action whose execution depends on what is done in the queue.

Operational test: If the supervisor closes the window, does anything stay pending? A feed has nothing to hold.

In my project: The chapter states the design principle plainly: externalize the authority mechanism, because agents do not reliably self-regulate and humans should not have to improvise oversight mid-incident. The queue logs entry time, first view, and decision time separately, which is what makes oversight quality measurable and not just oversight presence.

explanation panel

“Humans can edit it as a decision tree or flowchart, adding and removing nodes, which steers the agent rather than just interrogating it.”

website, CHART, the modalities

Definition: The channel through which an agent’s rationale reaches a human, with a short summary, a detailed view linked to the evidence, and an editable flowchart form.

Distinguishes it from: Explainability methods. The panel is content-agnostic: a rule trace, a confidence with evidence, a feature attribution and a language-model rationale all arrive through the same interface and are logged the same way. That is what makes explanation content a variable.

Operational test: Can the experimenter change the content, the format and the timing without changing the agent? If not, the explanation belongs to the agent and cannot be manipulated.

In my project: The edit direction is what separates this from every read-only rationale panel. Editing the tree steers the next decision, so the record contains not only what the human was shown but what the human wanted instead.

consultation pattern

Definition: How often a human opened an explanation, at what level of detail, at what moment relative to the decision, and whether opening it changed the decision.

Distinguishes it from: A self-report of how useful the explanation was. One is behaviour and

the other is an opinion about behaviour, and the over-reliance literature is the reason not to trust the second alone.

Operational test: Split approvals by whether the panel was opened first. Any difference in approval quality is the consultation effect.

In my project: The chapter names it as a logged stream with three fields: summary against detailed views, flowchart edits, and consultation timing relative to approvals.

communication modality

“Researchers can restrict visibility to simulate compartmentalization, cap message length, or inject misinformation”

website, CHART, the modalities

Definition: Public channel, private messages and mentions, with the system posting key events automatically and the agent posting status and asking clarifying questions.

Distinguishes it from: An informational edge, which governs what an agent may see. The chat governs what a teammate chooses to say. Compartmentalization can be imposed on either, and imposing it on the chat leaves the observation untouched.

Operational test: Is the restriction on the observation feed or on the message routing? Different edge, different modality.

In my project: The misinformation injection is the part with no counterpart in the graph. It is the only mechanism in CHART that puts a false statement in front of a teammate, which makes it the natural place to study whether a structural safeguard survives a lying channel.

1.4 The record

Input-Process-Outcome cycle

“Data is organized along the Input-Process-Outcome cycle.”

website, CHART, what the traces are for

Definition: The organizing scheme for the record. Inputs are session setup, agent selection and graph configuration. Processes are approvals, explanation consultations, chat and execution under dependency checks. Outcomes are mission results, trust and workload signals, and behaviour that feeds forward into the next mission. The chapter writes it IPOI and takes it from the team science lineage it cites, Cooke, Gorman, Myers and Duran 2013 and Salas, Reyes and McDaniel 2018.

Distinguishes it from: A three-phase timeline. The feedback arrow is what makes it a cycle: an outcome of one mission is an input to the next, so trust measured after mission one is a pre-task condition for mission two.

Operational test: Does anything in the Outcomes column reappear in the Inputs column of the following session? Without that, the scheme is a timeline.

In my project: Figure 13 and Table 2 of the chapter. The four streams are lobby and configuration, approval, explanation, communication, and execution, each with its own research

use and its own training use.

the linked trace

“The value is in the linking, not the individual streams.”

website, CHART, what the traces are for

Definition: The property that entries in different streams resolve against each other, so a coordination sequence arrives as one connected object.

Distinguishes it from: Synchronized clocks. Common timestamps let you sort events. Cross-stream identifiers let you say which explanation belonged to which approval, which is the claim that carries the causal questions.

Operational test: In a session with two humans and two agents acting concurrently, can the sequence be reconstructed without a person deciding which event goes with which? That concurrency case is where timestamps alone fail.

In my project: The worked sequence is the same in all three of my sources: the agent proposes isolation, the human opens the explanation, asks a clarifying question in chat, modifies the proposal to quarantine, approves the revision, and the system checks that scanning already happened. Microsecond timestamps plus cross-stream identifiers are what make it one object.

dependency check

Definition: The runtime test the environment performs on every submitted action against the edges into that agent, whose result is recorded whether it passes or fails.

Distinguishes it from: Validation of a malformed action. A dependency check rejects a well-formed action because of the structure, and the rejection is data.

Operational test: Is a failed attempt recorded as an event, or is it as if it never happened? Recorded is a check, discarded is a filter.

In my project: The chapter’s execution stream lists dependency checks satisfied or failed alongside synchrony, pool and temporal violations and retries. Those violations are the direct evidence for the Challenge II requirement, that a failure be attributable to the structure.

intention against executed action

Definition: The recording of what an agent or a human meant to do separately from what actually executed, so a modification is visible as a pair rather than as a single revised action.

Distinguishes it from: An action log with an edit history. The intention is captured at submission, before the structure has had its say, which is the only moment at which the counterfactual is available.

Operational test: Given a modified approval, can you recover the original proposal without diffing two snapshots?

In my project: Carried in the trace schema as separate fields for the intended action and target and the executed action and target, on both the human and the agent side, per round. This is the field pair that turns approvals into preference data, because the denial names the boundary and the modification names the alternative that was preferred.

preference data

“Approvals mark acceptable behaviour, denials mark boundaries, modifications identify the preferred alternative”

website, CHART, what the traces are for

Definition: The reuse of the same record as supervision for training an adaptive teammate, without a second collection instrument.

Distinguishes it from: Ratings collected for the purpose. Preference data here is a by-product of doing the task, so it carries the structure that produced it: the configuration trace says which authority boundaries the behaviour was generated under.

Operational test: Was the signal produced by an action the person was taking anyway? If they were asked, it is a rating.

In my project: Stated with an explicit priority, and the priority is the part to keep: reproducible and auditable logs first, learning layered on top as an optional consumer of the same data. The training use never gets to shape the instrument.

2 Working

white-box testbed

Definition: A platform whose internal structure is visible to and settable by the experimenter, as against one that exposes only a task and a score.

Distinguishes it from: Open source. Readable code is not the same as a settable structure; the property that matters is that the team specification is an input.

structure by interaction experiment

Definition: A design that crosses a taskwork configuration with a modality configuration, so the question is whether an interaction effect on trust or workload survives a change in the structure.

Distinguishes it from: Varying one and holding the other. That is the one-thing-at-a-time discipline for reading a single effect. The crossed design is what tests whether the effect generalizes across organizations.

the four read-outs

Definition: Quality, time, error and human load, the reproducible comparisons the two layers exist to support, with resilience and trust as the emergent states read alongside them.

Distinguishes it from: Mission success. Success is one component of quality and it is the one that hides everything else, which is the same case CyberAgentFlow makes against success rate on the single-agent side.

escalation chain

Definition: The ordered list of approvers an unanswered request passes to. An optional input on a control edge.

Distinguishes it from: The timeout policy, which decides the fate of a request nobody answers. The chain decides who is asked next; the policy decides what happens when the chain is exhausted.

risk-tier gating

Definition: Making the approval requirement conditional on a declared risk tier of the action rather than on its name, so that only irreversible or destructive actions require sign-off.

Distinguishes it from: Listing actions individually in `approval_required`. The gate generalizes to actions that did not exist when the condition was written.

compartmentalization

Definition: The operational practice CHART's informational edges and chat visibility rules are modelled on: roles differ in what they may see, and a junior analyst sees alert streams where a senior responder integrates across sources.

Distinguishes it from: Partial observability in the environment sense, which is a property of the sensors. Compartmentalization is imposed by the organization on top of whatever the sensors deliver.

CybORG and the CAGE challenges

Definition: The simulated cyber-defense environments and the associated challenge series CHART is defined against, which benchmark reinforcement learning agents against scripted adversaries and place the human outside as an evaluator.

Distinguishes it from: The Team Defense Game, which adapted CybORG to bring a human defender inside, first in direct control and later alongside an AI teammate that produced recommendations. CHART is the third step, and the step is making the coordination pattern itself settable.

3 Peripheral

interdependence typology

Definition: The organizational-theory lineage the five types are argued from: Thompson, *Organizations in Action*, 1967, and Burke and Salas on teamwork, 2003. Knowing that the five are an operationalization of an existing typology, not a fresh taxonomy, is enough.

interactive team cognition

Definition: Cooke, Gorman, Myers and Duran, *Cognitive Science* 37, 2013. The position that team cognition is in the interaction rather than in the members' heads, which is the justification for measuring teamwork through communication and coordination traces rather than through elicited mental models.

adaptive autonomy

Definition: The proposal that authority allocation should shift with the situation: routine decisions delegated, uncertain or high-risk decisions escalated. Cited in the chapter through Parasuraman and Riley 1997 and Chen, Gonzalez and Xu 2023.

Distinguishes it from: What CHART implements today, which is a fixed graph for the duration of a session. Adaptive autonomy is the third open question on the Next page, and it is a design commitment the platform has not yet made.

the transfer domains

Definition: Emergency response, healthcare and autonomous vehicle coordination, named as the high-stakes interdependent domains the same dependency types and modalities are claimed to describe. Cybersecurity is the motivating case, not the boundary.

4 Four sources, four emphases

CHART is written down in four places and each one carries something the others do not. A file that quotes only one of them will be missing a load the others carry.

The research page carries the five dependency types and the four modalities in full, the contribution claim, and the sentence that fixes the scope: the contribution is not the graph formalism, it is using the graph to vary authority distribution, information flow and sequencing across experiments.

The artifact pages carry the software. Two sentences appear nowhere else. The design page states that because the structure is a DAG it can be walked and queried rather than only read, and that authority becomes a property of the graph, so an action on a control edge can be approved only by the teammate on the other end of it however sensible it looks to anyone else. The use page states the dual role, an experimental platform and a structured data generator at once, and gives the five single named changes and the microsecond-timestamp reconstruction guarantee.

The book chapter carries the argument and the parameters. Table 1 is the only place the required and optional inputs of each dependency type are written down. Table 2 is the only place the data streams are set against their research and training uses. It also carries the reason each modality exists, stated as a gap in what agents can do for themselves: agents that cannot request permission, agents that defer on everything, agents that produce fluent rationales that are not faithful, agents that cannot track mention targets in a multi-party conversation.

The code carries what is actually built. Seven `dependency_type` values against the page's five, the observation filters composed at configuration time, the pool threshold as a set of required contributions rather than a count, and the trace schema with intention and execution as separate fields.

The chapter's second revision round turned on a single demand, and it is the demand this file is organized around: show how a finding is extracted from the graph, rather than claiming what the graph would let you find. Three analytical examples had to name their graph-based method, the DAG definition had to state its node types and what the representation buys for post-session analysis, and the sections that restated general human-AI teaming concepts had to give up their space to make room. Every entry above is written to survive the same demand.

5 Next

The artifact page states the position plainly and it is the right one.

“CHART so far captures the syntax of human-agent teamwork, and whether that syntax is complete

or succinct has not been tested.”

artifacts, CHART, what the next version has to answer

Three questions follow from it, and they are ordered by how much they cost.

Complete. Can the graph express how a security operations centre or an incident response team actually works? This is answerable now, and cheaply, by taking one written incident response procedure and attempting to encode it. Every constraint that will not go into an edge is either a missing type or evidence that five is enough. The two composed types already in the code are the first candidates.

Prescriptive. Can teamwork written in CHART be prescribed and enforced as well as recorded? The control, temporal and pool edges already enforce. The gap is that a team’s own intended protocol is not currently expressible as anything the system checks it against, so there is no notion of the team deviating from its own plan.

Diagnosable. Can a session be monitored while it runs and diagnosed afterwards, so a structural fault surfaces while it is still correctable? This is the one with the design problem inside it. A monitor that watches the graph during a mission is an authority in the session, and nothing in the architecture says who it reports to.

Succinctness is the fourth and nobody has asked it yet. Two graphs can enforce identical constraints and differ in how many edges they take, and if the count moves with the encoding rather than with the team, then edge count is not a measure of coordination demand and no result should be read as if it were.